



Lesson 3

Programming Styles and Objects



A Programming Style is a set of rules or guidelines used when writing the source code for a computer program.

Imperative programming describes **how** to do

Ex. “Walk me to my table with menus, napkins and silverware, set the table and alert the server.

Types of Imperative Programming

- Procedural programming
- Object oriented programming

```
let array = [1, 2, 3, 4, 5, 6]
var sum = 0
var odds = []

for element in array {
    sum += element
    if element % 2 == 1 {
        odds
    }
    .append(element)
}
```

Declarative programming describes **what** to do. Ex. “Table for one.”

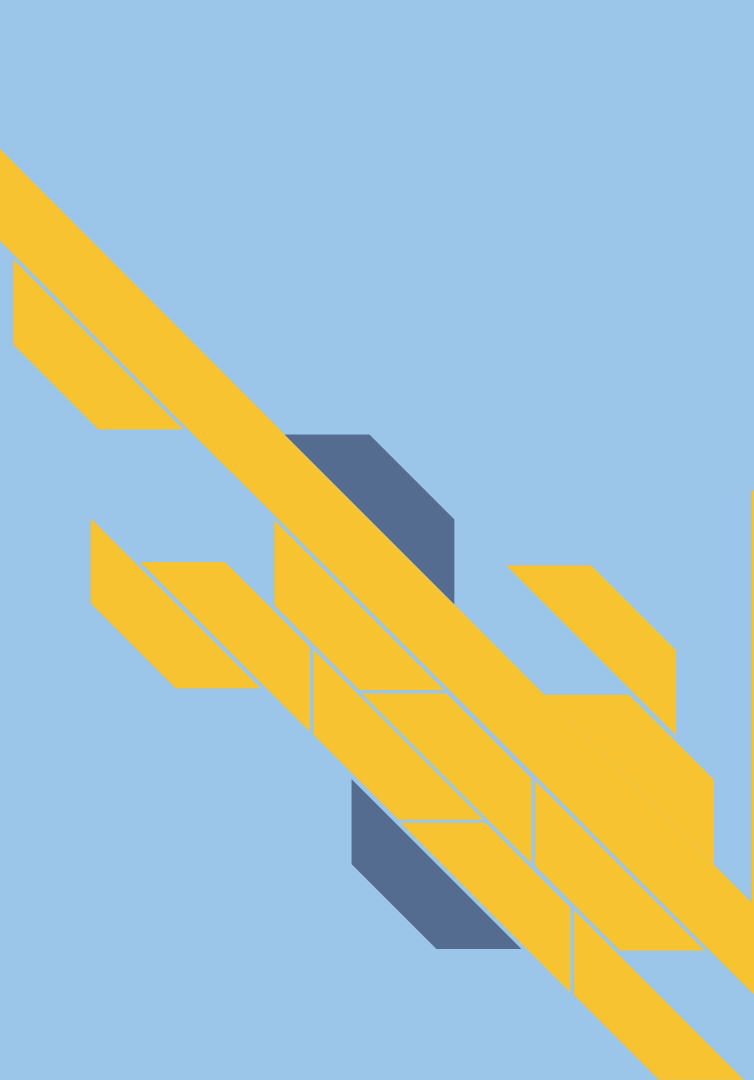
Types of Declarative Programming

- Database languages (Structured Query Language - SQL)
- Functional programming
- Logical programming

```
let numbers = [1, 2, 3, 4, 5, 6]
let sum = reduce(numbers, 0, +)
let odds = filter(numbers, { $0 % 2 == 1 })
```



Types of Declarative Programming





Structured Query Language (SQL) is an **declarative** language used to create and manipulate data in a database.

```
SELECT CustomerName, City FROM Customers  
WHERE Country='Mexico';
```

```
UPDATE Supplier SET City = 'Sydney' WHERE  
Name = 'Pavlova, Ltd.'
```

```
DELETE FROM Product WHERE UnitPrice > 50
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

```
INSERT INTO Customer (FirstName, LastName, City, Country, Phone)  
SELECT LEFT(ContactName, CHARINDEX(' ',ContactName) - 1),  
        SUBSTRING(ContactName, CHARINDEX(' ',ContactName) + 1,100),City, Country, Phone  
FROM Supplier  
WHERE CompanyName = 'Bigfoot Breweries'
```



Functional programming

Functional Programming is a style of programming that emphasizes the **evaluation of expressions** rather than the execution of commands.

This means that a program, written in functional programming, consists of a set of functions that always return the same values. The variables in this style don't change value.

Basically, its programming without assignment statements like `x=5`!

```
from random import random

def move_cars(car_positions):
    return map(lambda x: x + 1
               if random() > 0.3 else
               x, car_positions)

def output_car(car_position):
    return '-' * car_position

def run_step_of_race(state):
    return {'time': state['time'] - 1,
            'car_positions': move_cars(state
                                         ['car_positions'])}

def draw(state):
    print ''
    print '\n'.join(map(output_car, state
                        ['car_positions']))

def race(state):
    draw(state)
    if state['time']:
        race(run_step_of_race(state))

race({'time': 5, 'car_positions': [1, 1,
1]})
```

Logic programming

Logic Programming is programming based on the notion of logical deduction. It is programming statements that express facts and rules about problems within a system of formal logic. Rules are written as logical clauses with a head and a body and they you submit queries about those facts and rules.

```
4 tree(if_then_else('is clever',
5     if_then_else('color green',
6         house(slytherin),
7         if_then_else('color blue',
8             house(ravenclaw),
9             false)),
10    if_then_else('is brave',
11        if_then_else('color red',
12            house(gryffindor),
13            if_then_else('color yellow',
14                house(hufflepuff),
15                false)),
16        false))),
17    false))).
18
19 mtree(if_then_else('fur',
20     if_then_else('color red',
21         mascot(lion),
22         if_then_else('color yellow',
23             mascot(badger),
24             false)),
25     if_then_else('not fur',
26         if_then_else('color green',
27             mascot(serpent),
28             if_then_else('color blue',
29                 mascot(eagle),
30                 false)),
31         false))),
32     false))).
33
```

```
33
34 house(A) :-
35     tree(T),
36     tree_house(T, A).
37
38 mascot(B) :-
39     mtree(T),
40     tree_mascot(T,B).
41
42 tree_house(house(A), A).
43 tree_house(if_then_else(Cond,Then,Else), A) :-
44     ( is_true(Cond) ->
45         tree_house(Then, A)
46     ; tree_house(Else, A)
47     ).
48
49 tree_mascot(mascot(B), B).
50 tree_mascot(if_then_else(Cond,Then,Else), B) :-
51     ( is_true(Cond) ->
52         tree_mascot(Then, B)
53     ; tree_mascot(Else, B)
54     ).
55
```

house(A).

is clever?

is brave?

color red?

color yellow?

A = hufflepuff

house(A).

is clever?

color green?

A = slytherin

house(A).

is clever?

color green?

color blue?

A = ravenclaw

mascot(B).

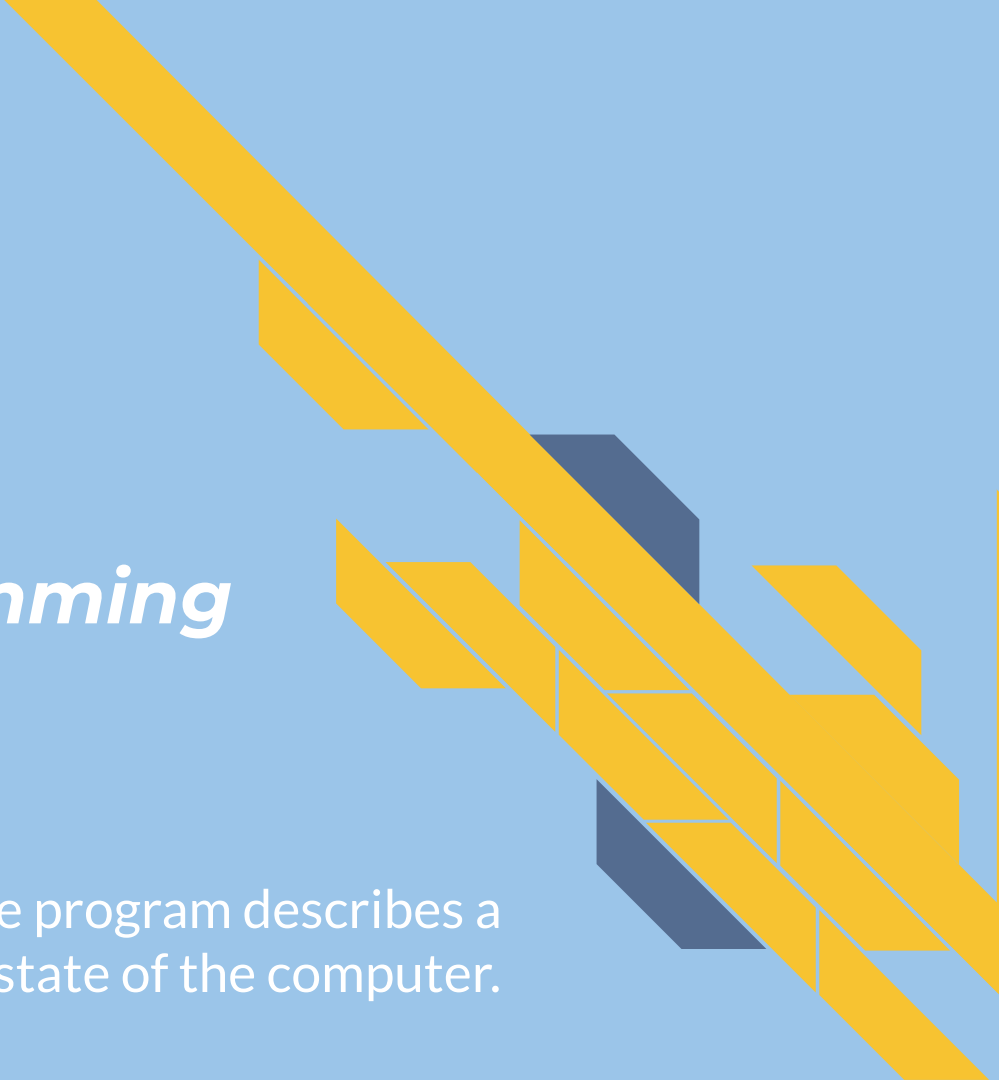
fur?

color red?

color yellow?

B = badger

?- mascot(B).



Types of Imperative Programming

Imperative Programming is when the program describes a sequence of steps that change the state of the computer.

Imperative programming

is a procedural language because the code is organized into procedures like:

- blocks
- functions
- modules
- packages

```
#!/usr/bin/python
import os
import sys
import string
import optparse

...
from glideinwms.lib import condorExe
# Main function
def main(argv):
    feconfig=frontenvparse.FEConfig() # FE
    configuration holder
    ... # parse arguments
    feconfig.config_optparse(argparser)
    (options, other_args) =
    argparser.parse_args(argv[1:])

    if len(other_args)<1:
        raise ValueError, "Missing glidein_name"
    glidein_name = other_args[0]
    if len(other_args)>=2:
        log_type=other_args[1]
    else:
        log_type="STARTD"
    ...
    return 0
# S T A R T U P
if __name__ == '__main__':
    try:
        sys.exit(main(sys.argv))
    except Exception, e:
        sys.stderr.write("ERROR: Exception msg
%s\n"%str(e))
        sys.exit(9)
```

Objects (Object Oriented Programming)

is based on the concept of "objects", which can contain data, in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).

Some important properties of OOP Programming!

- Abstraction
- Encapsulation
- Polymorphism
- Composition
- Inheritance
- Delegation

Classes are blueprint to create an object

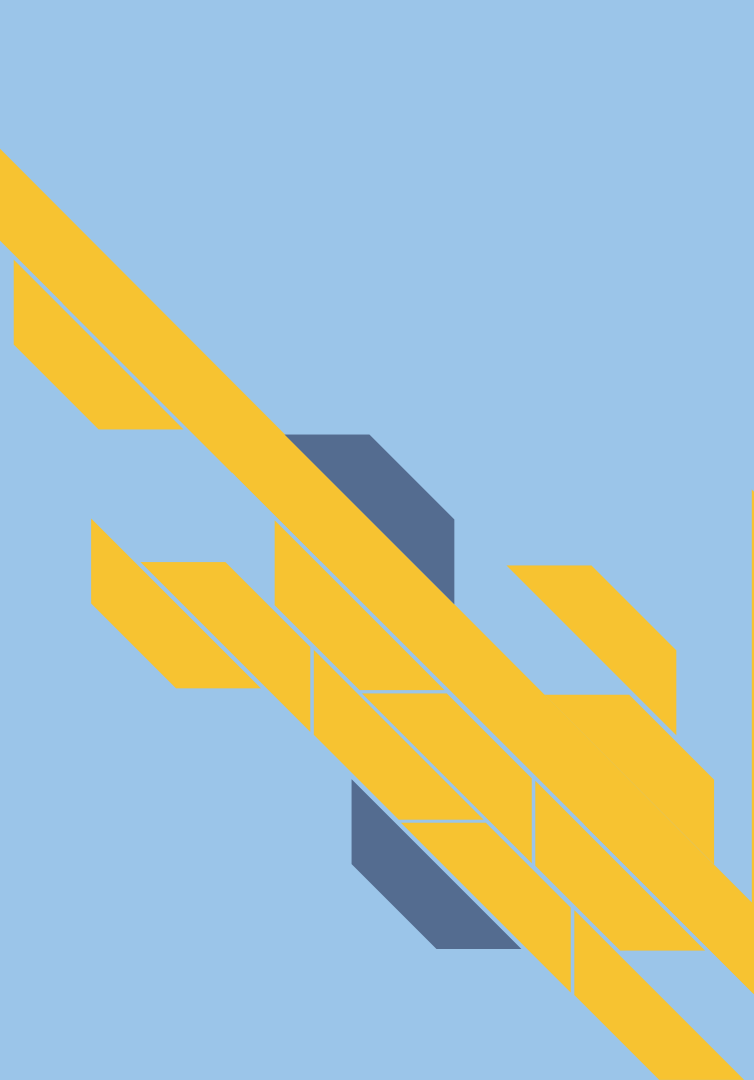
```
1 class Person:
2     firstName = " "
3     lastName = " "
4     age = None
5     gender = " "
6     race = " "
7
8     #these are methods or functions that are unique to Person
9     #! parentheses are placeholders that describe what you should put in to declare a person
10    def __init__(self, first, last, age, gen, race):
11        #here we use these placeholders w/ data you will enter and set our variables to them
12        self.firstName = first
13        self.lastName = last
14        self.age = age
15        self.gender = gen
16        self.race = race
17
18    def celebrateBirthday(self):
19        print("Happy Birthday!!")
20        self.age += 1
21
22    #this is called a getter. Getters get information from your object
23    def getName(self):
24        return self.firstName + " " + self.lastName
25
26    #this is called a setter. This sets/overrides information in your object
27    def changeFullName(self, new_first, new_last):
28        if new_first != None and new_first != " ":
29            self.firstName = new_first
30
31        if new_last != None and new_last != " ":
32            self.lastName = new_last
33
34    #this is a new object of type Person
35    pete = Person("Pete", "Johnson", 24, "male", "Croatian")
36
37    print(pete.getName())
38    pete.changeFullName("Joe", "Joeson")
39    print(pete.getName())
40    pete.celebrateBirthday()
41
42
43
44
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL 2: Python +

```
PS C:\Users\miric> & python "c:\Users\miric\Documents\Important DOCS\Fermlab_POFs\package_ex.py"
Pete Johnson
Joe Joeson
Happy Birthday!!
```



Important Properties of Object Oriented Programming (OOP)



Abstraction

Abstraction is the concept of exposing only the required essential characteristics and behavior with respect to a context.

- ✓ Full Name
- ✓ Address
- ✓ Contact Number
- ✓ Tax Information
- ✓ Favorite Food
- ✓ Favorite Movie
- ✓ Favorite Actor
- ✓ Favorite Band

okay, we might not need all these customer information for a banking application

Basically, it's the idea that the user doesn't need to know what's happening under the hood. If you're using a calculator you don't need to know how pressing the button sends a signal to the calculator - you just have to press the button.

It's also the idea that one part of your program doesn't need *all* the information of another part. A function that withdraws money from the user's account doesn't need to know the user's name or their favorite food.

Encapsulation

Encapsulation is the concept of hiding data from other parts of your program in order to moderate access. This is achieved by making this **private** or **public**.

Things that are **private** can only be accessed inside the scope of where they were created.

Things that are **public** can be accessed from anywhere in the program.

How it's declared in Python



Name	Notation	Behavior
name	Public	Can be accessed from inside and outside
_name	Protected	Like a public member, but they shouldn't be directly accessed from outside
__name	Private	Can't be seen and accessed from outside

```
1 class Person:
2     firstName = ""
3     lastName = ""
4     __age = None
5     gender = ""
6     race = ""
7
8     #these are methods or functions that are unique to Person
9     #! parentheses are placeholders that describe what you should put in to declare a person
10    def __init__(self, first, last, age, gen, race):
11        #here we use these placeholders w/ data you will enter and set our variables to them
12        self.firstName = first
13        self.lastName = last
14        self.age = age
15        self.gender = gen
16        self.race = race
17
18    def celebrateBirthday(self):
19        print("Happy Birthday!!")
20        self.age += 1
21        self._checkAge #checkAge can be used IN the function but NOT outside
22
23    def _checkAge(self): #in python, to say something is private you do underscores on either side
24        if self.age > 20:
25            print("Congrats! You are able to vote and drink!")
26        elif self.age > 17 and self.age < 21:
27            print("Congrats! You are able to vote!")
28        else:
29            print("You are a minor!")
30
31    #this is called a getter. Getters get information from your object
32    def getName(self):
33        return self.firstName + " " + self.lastName
34
35    #this is called a setter. This sets/overrides information in your object
36    def changeFullName(self, new_first, new_last):
37        if new_first != None and new_first != " ":
38            self.firstName = new_first
39
40        if new_last != None and new_last != " ":
41            self.lastName = new_last
42
43    #this is a new object of type Person
44    pete = Person("Pete", "Johnson", 24, "male", "Croatian")
45    pete.celebrateBirthday()
46    print("Now let's check your age again!")
47    pete._checkAge() #this will not print because this method is not available outside of the function
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL

PS C:\Users\mlrinc> python "c:/Users/mlrinc/Documents/Important DOCS/Ferilab_PDFs/package_ex.py"

Happy Birthday!!

Now let's check your age again!

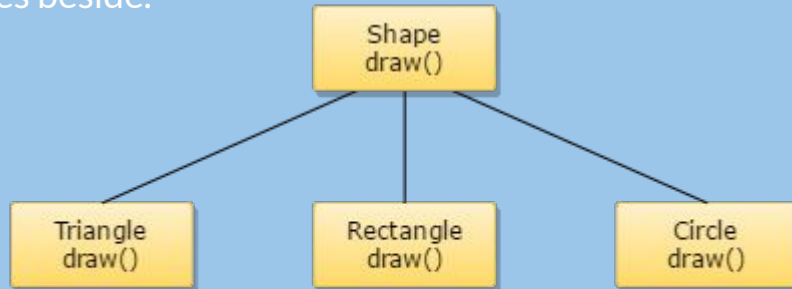
Congrats! You are able to vote and drink!

Polymorphism

Polymorphism is the process of representing one form in multiple forms

Suppose if you are in class room that time you behave like a student, when you are in market at that time you behave like a customer, when you are at your home at that time you behave like a son or daughter, Here one person present in different-"different behaviors".

In code, one method/function can have multiple actions such as the images beside.



Polymorphism



Inheritance

Inheritance is receiving something from the previous holder. In inheritance, there is a superclass and a subclass also called a parent class and a child class **also** called a base class and derived class.

Ex. Father gives his property to child , father got that properties from child's grandfather , so child is the taker and father is giver , hence father works as a base class and child as derived class



```
class Person(object):
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def show(self):
        print 'Person Name: %s' %(self.name)
        print 'Person Age: %s' %(self.age)
class Employee(Person):
    def __init__(self,name,age,company,sal):
        self.company=company
        self.sal=sal
        super(Employee,self).__init__(name,age)
    def __repr__(self):
        return str
        (self.name+self.age+self.company+self.sal)
```

In this example, *Person* is the base class and *Employee* is derived class. *Employee* inherits name and age from *Person*

To check your inheritance ask yourself “Are all employees people?” Yes. “Are all people employees?” No.

Ex. shapes.py

Canvas class

Defines a frame buffer

Shape class

Abstract base class for shapes. All Shapes can paint themselves on a canvas

Child

Inherits from Shape, implements paint() and the constructor __init__()

```
class Canvas:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.data = [[' ']*width for i in range(height)]

    def setpixel(self, row, col):
        self.data[row][col] = '*'

    def getpixel(self, row, col):
        return self.data[row][col]

    def display(self):
        print("\n".join(["".join(row) for row in self.data]))

class Shape:
    def paint(self, canvas): pass

class Line(Shape):
    def __init__(self, x1, y1, x2, y2):
        self.x = x1
        self.y = y1
        self.w = x2 - x1
        self.h = y2 - y1

    def paint(self, canvas):
        ratio = self.w / self.h
        if self.w >= self.h:
            for i in range(self.h):
                canvas.setpixel(self.x+int(i*ratio), self.y+i)
        else:
            for i in range(self.w):
                canvas.setpixel(self.x+i, self.y+int(i/ratio))
```

```
# ... Continues from previous column
class Rectangle(Shape):
    def __init__(self, x, y, w, h):
        self.x = x; self.y = y; self.w = w; self.h = h

    @staticmethod
    def hline(x, y, w, canvas):
        i = 0
        while i <= w:
            canvas.setpixel(x+i, y); i += 1

    @staticmethod
    def vline(x, y, h, canvas):
        i = 0
        while i <= h:
            canvas.setpixel(x, y+i); i += 1

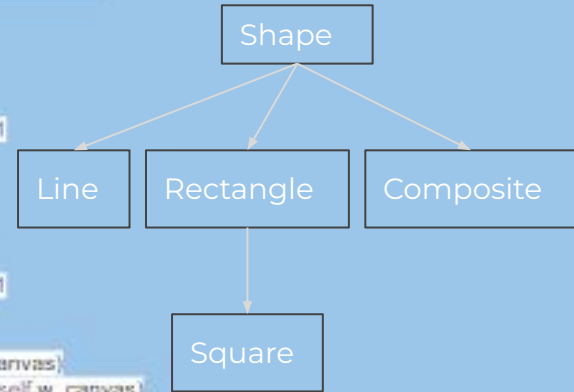
    def paint(self, canvas):
        self.hline(self.x, self.y, self.w, canvas)
        self.hline(self.x, self.y + self.h, self.w, canvas)
        self.vline(self.x, self.y, self.h, canvas)
        self.vline(self.x + self.w, self.y, self.h, canvas)

class Square(Rectangle):
    def __init__(self, x, y, size):
        Rectangle.__init__(self, x, y, size, size)

class CompoundShape(Shape):
    def __init__(self, shapes):
        self.shapes = shapes

    def paint(self, canvas):
        for s in self.shapes:
            s.paint(canvas)
```

Inheritance of shapes.py



Inheritance (2)

Square is a Rectangle with height = width
Gets paint() from Rectangle



Ex2. shapes.py

Import shapes

All classes are defined in the module shapes.py

Instantiate the shapes

Creating objects from all the shapes using the **CompoundShape** to paint all at once on **mycanvas**

NOTE: how you use the objects (casts). The classes (molds) are used only to instantiate new objects

```
# Import all the shapes
from shapes import *
```

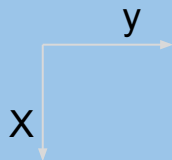
```
# Create a canvas
mycanvas = Canvas(20, 20)
```

```
# Draw a rectangle, a
square and a line on the
canvas
r1 = Rectangle(2, 3, 4, 5)
s1 = Square(4, 4, 6)
l1 = Line(5, 2, 15, 15)
shapes = [r1, s1, l1]
c1 = CompoundShape(shapes)
c1.paint(mycanvas)
```

```
# Show the result
mycanvas.display()
```

Instantiate a Canvas

Creating an object of class (type) Canvas, a frame buffer



Display the result

The Canvas mycanvas is printed on the screen

```
>>> mycanvas.display()
```

```
*****
*       *
*****
*** * *
***** *
*       *
** * *
* * *
*****
      *
        *
          *
```



Alright! Let's Get
Back to Coding!